

Design Overview for Graphing Calculator

Name: Ryan Weber

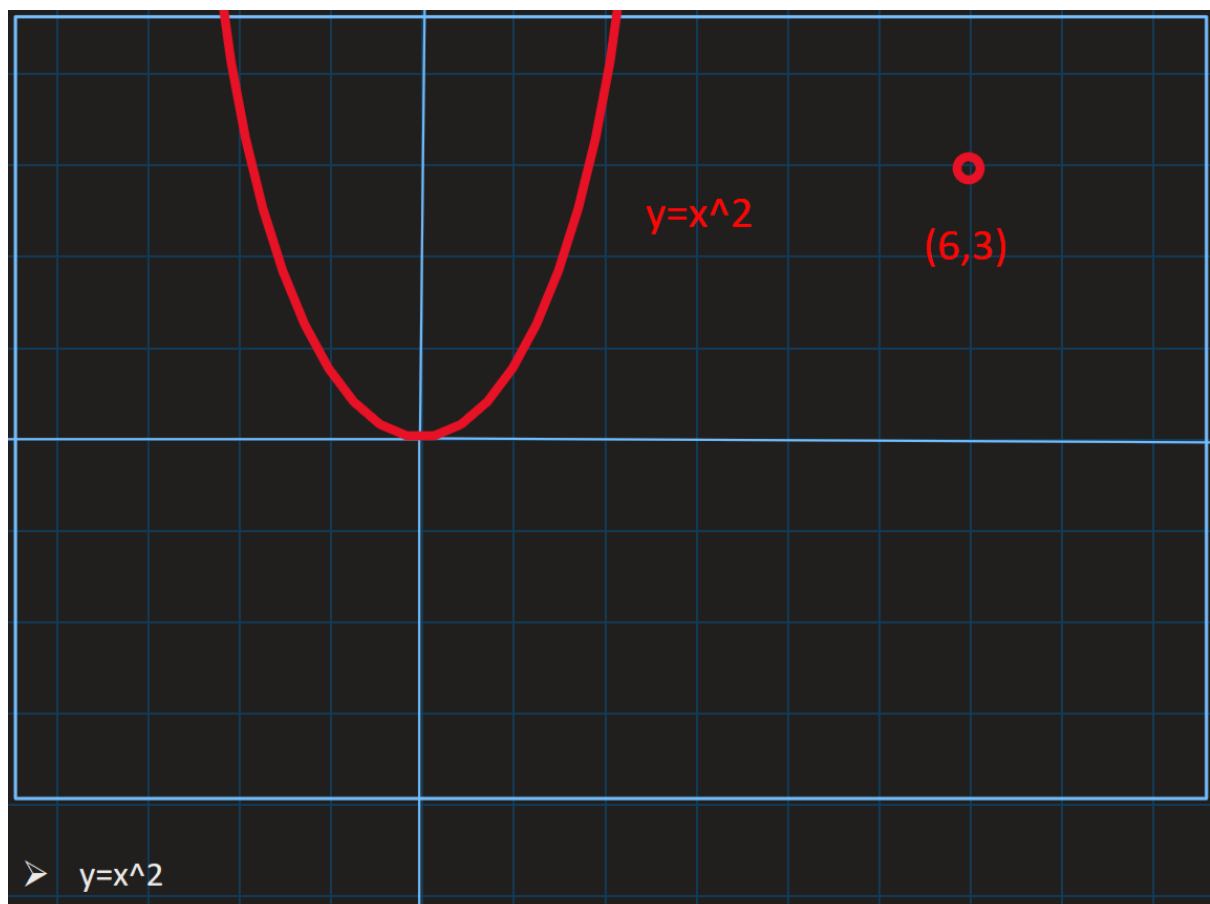
Student ID: 105914892

Summary of Program

The program is a graphing calculator, which takes mathematical equations as an input from the user through the terminal, which could be expanded to a user interface, and displays graphs and points the screen as output. The program can draw many types of graphs based on the user's input, using different mathematical features like brackets and exponents. The user can also input and view coordinates as individual points. The program could additionally calculate and display graph feature such as intersections, endpoints, or asymptotes. Saving and loading could also be added.

The graphic calculator uses a `GraphObject` class, which is inherited by other classes to specify the thing being displayed. For the main `Graph` class, the program will evaluate the coordinates of "nodes," which link together to produce a graph.

The program helps users visualise the structure of mathematical equations, helping to solve problems or building their understanding in mathematics.



Required Roles

This program could be expanded with more classes and functionality. Listed are core roles required for the program to work as a graphing calculator.

Table 1: Render – Holds graph objects and camera / rendering details

Responsibility	Type Details	Notes
camPos	Private, Vector2	Stores the location of the “camera” – where the user is viewing the plane from. Changing this changes the actual draw positions of graph objects. It is passed into the draw frame method.
graphObjects	Private, GraphObject List	Stores all graph objects which are to be rendered. Anything graph object wanting to draw to the screen must be in this list. Modifiable at runtime.
Render()	Public, Constructor	Makes a new Render with a (0, 0) camPos, and an empty list of graph objects. This may be populated from a save file.
CamPos	Public, Vector2, Property	Read and write property to CamPos
DrawFrame()	Public	Loops through every graph object in the list and calls DrawObject()
AddGraphObject()	Public, in _equation : string	Adds GraphObjects to the plane. Methods for removing and modifying GraphObjects would also be useful

Table 2: GraphObject – Objects to be drawn to the screen on the cartesian plane. Is a parent class.

Responsibility	Type Details	Notes
position	Private, Vector2	X Y position of this GraphObject. Not needed for graphs

colour	Private, Color	Colour used to display this graph object
GraphObject()	Public, Constructor	Makes a new graph object with a colour of red and a (0, 0) position
GraphObject()	Public, Constructor, in Vector2 : position	Makes a new graph object with a colour of red and a given position
GraphObject()	Public, Constructor, in _ position : Vector2, in _colour : Color	Makes a new graph object with a given colour and a given position
Position	Public, Vector2, Property	Read and write property to position
Colour	Public, Color, Property	Read and write property to colour
DrawObject()	Public, Virtual, in _camPos : Vector2	Is called by Render. Performs the appropriate draw action for that object. If it is not overridden, draws a dot at its position

Table 3: Graph – Graph object, connected by nodes and drawn with lines.

Responsibility	Type Details	Notes
equation	Private, string	Equation given from user input. Defines how the graph exists and is used to calculate nodes
nodes	Private, Node List	Stores nodes related to that graph, linking them together. This is changed at runtime based on camPos and perhaps cam size etc
Graph()	Public, Constructor	Makes a new graph, creating node and calling CalculateNodes().
DrawObject()	Public, Override, in Vector2 : camPos	Draws lines between nodes, rendering the graph
CalculateNodes()	Public, in _ equation : string	Calculates the position of each relevant node. Ran whenever the view is updated, perhaps every second, or when the camPos is changed. This may be an expensive operation.

CalculateNode()	Public, in <code>_equation</code> : string, in <code>_x</code> : float	Calculates the position of a node based on the graph equation. This could alternatively be done in the Node object
------------------------	--	--

Table 4: Point – Single displayed coordinate

This class does not need a position or other data as it is inherited from GraphObject.

Responsibility	Type Details	Notes
Point()	Public, Constructor	Creates a point

Table5: Node – Part of a graph used to link line.

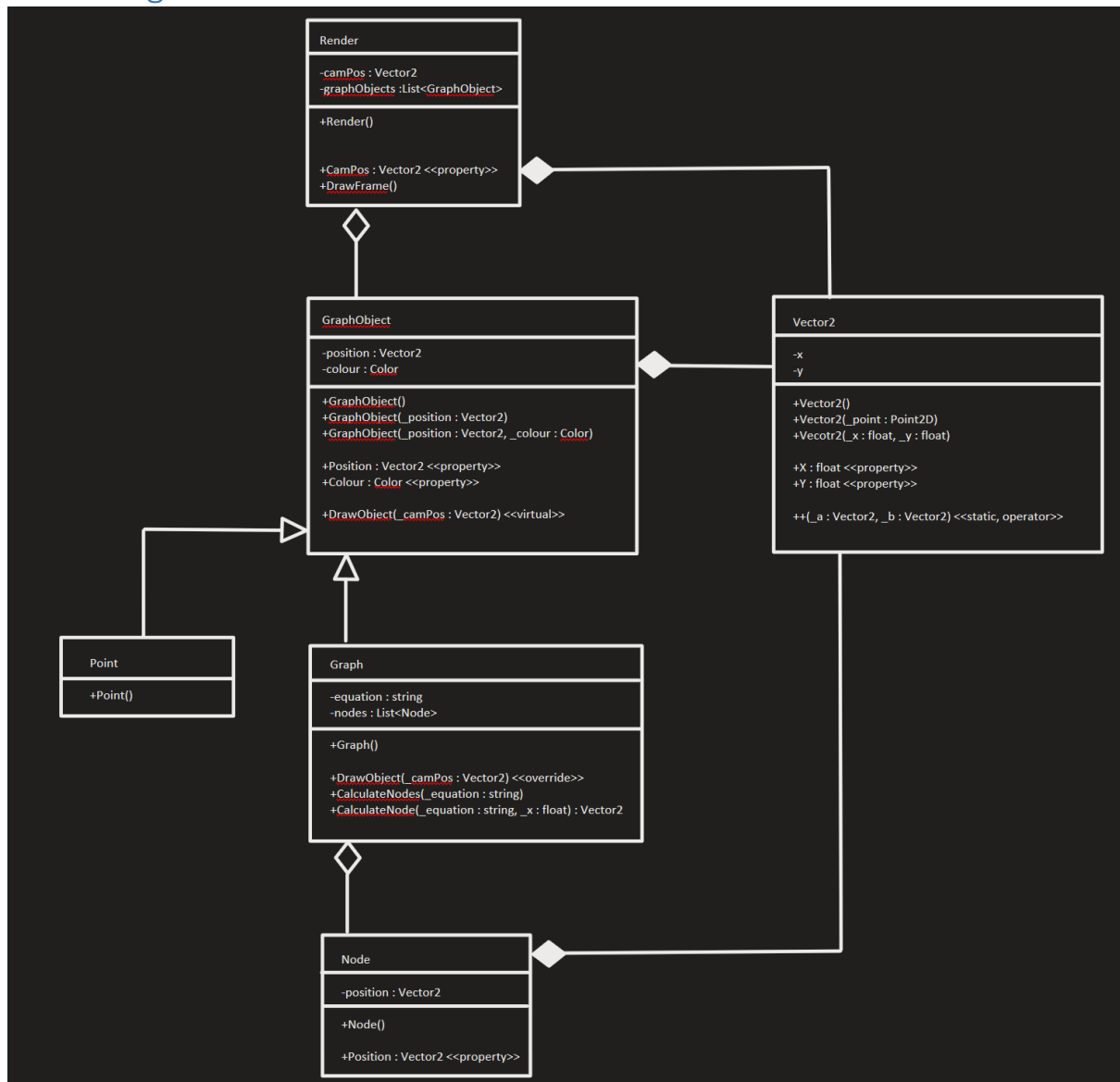
Although this is not displayed by itself, it may be beneficial to have it inherit from GraphObject in the future.

Responsibility	Type Details	Notes
position	Private, Vector2	Stores position in cartesian plane.
Node()	Public, Constructor	Creates a node
Position()	Public, Property, Vector2	Read and write property to position

Table6: Vector2 – Custom class for storing 2D coordinates.

Responsibility	Type Details	Notes
x	Private, float	X value of the coordinate
y	Private, float	Y value of the coordinate
Vector2()	Public, Constructor	Creates Vector2
Vector2()	Public, Constructor, in <code>_point</code> : Point2D	Creates Vector2 at an inputted Point2D
Vector2()	Public, Constructor, in <code>_x</code> : float, in <code>_y</code> : float,	Creates Vector2 at an inputted x and y coordinate
X	Public, Property	Read and write property to x
Y	Public, Property	Read and write property to y
++	Public, Static, Operator, in <code>_a</code> : Vector2, in <code>_b</code> : Vector2	Adds together 2 Vector2s

Class Diagram



Sequence Diagram

